

最优控制与强化学习大作业

HW0 HW1 HW2

姓名：李丰克

学号：3230105182

日期：2026 年 5 月 16 日

目录

1	HW0	3
1.1	Julia 的安装与配置	3
1.2	Question1: Julia 中的微分	3
1.2.1	单输入函数	3
1.2.2	多变量输入函数微分	3
1.2.3	复合函数微分	3
1.2.4	常见的 FD 错误	4
1.3	Question2: 牛顿迭代法	4
1.3.1	一维牛顿法	4
1.3.2	多变量牛顿法	4
1.3.3	优化中的牛顿法应用	5
2	HW1	6
2.1	Question1: 积分	6
2.1.1	显式积分	6
2.1.2	隐式积分	6
2.2	Question2: 等式约束优化	8
2.2.1	牛顿法	8
2.2.2	全牛顿法和高斯牛顿法	9
2.2.3	平衡四足机器人	9
2.3	Question3: 对数域内点二次规划 (QP) 求解器	10
2.3.1	KKT 条件	11
2.3.2	互补松弛条件	11
2.3.3	对数域变量代换	11
2.3.4	对数域求解器	11
2.3.5	用 QP 模拟下落的砖块	12
2.3.6	下落砖块的 QP 问题表述	12
2.3.7	砖块模拟	13
2.3.8	求解 QP 问题	14
3	HW2	14
3.1	Question1: 有限时间步 LQR	14
3.1.1	模型离散化	14
3.1.2	基于凸优化的有限时间步 LQR	14
3.1.3	基于 Riccati 递归的有限时间步 LQR	15
3.1.4	LQR 的两个优势	16
3.1.5	无限时间步 LQR	16
3.2	Question2: 小车倒立摆模型的 LQR	16
3.2.1	围绕平衡点的无限时间步 LQR	17
3.2.2	无限时间步 LQR 的吸引域	17
3.2.3	无限时间步 LQR 的代价函数调参	18
3.2.4	用于轨迹跟踪的时变 LQR (TVLQR)	18

3.3	Question3: 最优交会与对接	18
3.3.1	有限时间步 LQR	19
3.3.2	凸轨迹优化	19
3.3.3	凸模型预测控制	20

1 HW0

1.1 Julia 的安装与配置

在终端中用 `winget install julia -s msstore` 和 `juliaup add lts` 安装 julia 的 LTS 版本, 随后在 `vscode` 中安装 julia 和 `jupyter` 拓展, 最后在 `notebook` 中选择 julia LTS 内核即可。

1.2 Question1: Julia 中的微分

Julia 中自带一个用于前向微分的包 `ForwardDiff`, 其可用于单输入函数、多输入函数、复合函数等场景, 要用到的函数如下所示。

```
1 FD.derivative(f,x)#标量或向量值函数 f
   关于标量 x 的导数 (derivative)
2 FD.jacobian(f,x)#向量值函数 f 关于向量
   x 的雅可比矩阵 (jacobian)
3 FD.gradient(f,x)#标量值函数 f 关于向量
   x 的梯度 (gradient)
4 FD.hessian(f,x)#标量值函数 f 关于向量
   x 的海森矩阵 (hessian)
```

对任一个函数 $f(x): \mathbb{R}^N \rightarrow \mathbb{R}^M$, 雅可比矩阵定义为:

$$\frac{\partial f(x)}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

对一个标量函数 $f(x): \mathbb{R}^N \rightarrow \mathbb{R}$, 雅可比矩阵是一个行向量:

$$\frac{\partial f(x)}{\partial x} = \left[\frac{\partial f_1}{\partial x_1} \quad \cdots \quad \frac{\partial f_1}{\partial x_n} \right]$$

其转置被称为梯度:

$$\nabla f(x) = \left[\frac{\partial f(x)}{\partial x} \right]^T$$

1.2.1 单输入函数

```
1 function foo4(x)
2     Q = diagm([1;2;3.0])
3     return 0.5*transpose(x)*Q*x/x[1] -
         log(x[1])*exp(x[2])^x[3] #
4 end
5
6 function foo4_expansion(x)
```

```
# TODO: calculate the gradient of
foo4 evaluated at x
g = FD.gradient(foo4, x)
# TODO: calculate the hessian of
foo4 evaluated at x
H = FD.hessian(foo4, x)
return g, H
end
```

如上所述是调用 `FD` 库计算梯度和海森矩阵的方法。

1.2.2 多变量输入函数微分

```
function eulers(x,u,J)
    ẋ = J\u - cross(x,J*x)
    return ẋ
end

function eulers_jacobians(x,u,J)
    # TODO: fill in the following two
    jacobians
    A = FD.jacobian(dx -> eulers(dx,u,
J),x)
    B = FD.jacobian(du -> eulers(x,du,
J),u)
    return A, B
end
```

如上所述是调用 `FD` 库计算多变量函数微分的方法。

1.2.3 复合函数微分

```
1 function f2(x)
2     return x*sin(x)/2
3 end
4 function g2(x)
5     return cos(x)^2 - tan(x)^3
6 end
7
8 function composite_derivs(x)
9     #TODO: return y/ x where y=g2(f2(x))
10    return FD.derivative(dx -> g2(f2(
dx)),x)
11 end
```

用链式法则求微分。

1.2.4 常见的 FD 错误

ForwardDiff.dual 与 Float64 数据类型是不一致的，当将其强制塞入一个 Float64 向量时，如先创建一个全 0 向量再赋值，这样会报错。

解决方法一为不创建全零数组，直接创建一个 xdot，然后直接把计算的结果代入进去，编译器会自动计算出应该把 xdot 设为什么类型。

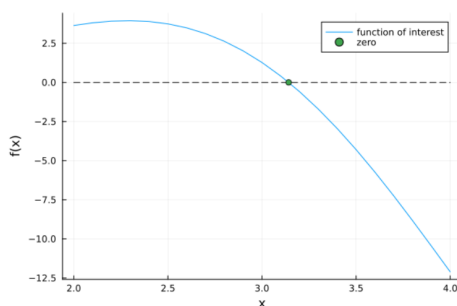
解决方法二为在创建全零向量中考虑进 x 的数据类型，如用 zeros(eltype(x),length(x)) 来实现，其会创建一个元素类型和 x 完全相同的全零向量。

```
1 # TODO: fix this error when trying to
2   diff through this function
3 function dynamics(x,u)
4     xdot = zeros(promote_type(eltype(x)
5     ),eltype(u)), length(x))
6     xdot[1] = x[1]*sin(u[1])
7     xdot[2] = x[2]*cos(u[2])
8     return xdot
9 end
```

1.3 Question2: 牛顿迭代法

1.3.1 一维牛顿法

考虑一个非线性函数的根。



使用牛顿法来求解这个根，定义残差函数：

$$r(x) = \sin(x)x^2$$

我们希望将这个残差函数驱动为零。为此，我们从初始猜测值 x_k 开始，并用一阶泰

勒展开式来近似我们的残差函数：

$$r(x_k + \Delta x) \approx r(x_k) + \left[\frac{\partial r}{\partial x} \right]_{x_k} \Delta x$$

现在我们需要找到这个线性近似函数的根，即找到一个 Δx ，使得 $r(x_k + \Delta x) = 0$

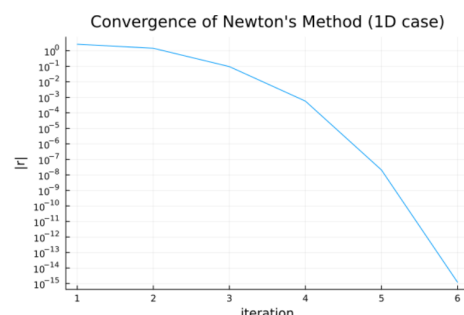
$$\Delta x = - \left[\frac{\partial r}{\partial x} \right]_{x_k}^{-1} r(x_k)$$

然后再不断迭代更新根的估计值：

$$x_{k+1} = x_k + \Delta x$$

```
1 function newtons_method_1d(x0::Float64
2   , residual_function::Function;
3   max_iters = 10)::Vector{Float64}
4   X = zeros(max_iters)
5   X[1] = x0
6   for i = 1:max_iters
7       # TODO: Newton's method here
8       r = residual_function(X[i])
9       if abs(r) < 1e-10
10         return X[1:i]
11       end
12       J = FD.derivative(
13         residual_function, X[i])
14       X[i+1] = X[i] - J\r
15   end
16   error("Newton did not converge")
17 end
```

如图是牛顿迭代法的代码实现。注意在代码实现中一般为计算效率不求逆矩阵，而是用解线性方程组来求解。



1.3.2 多变量牛顿法

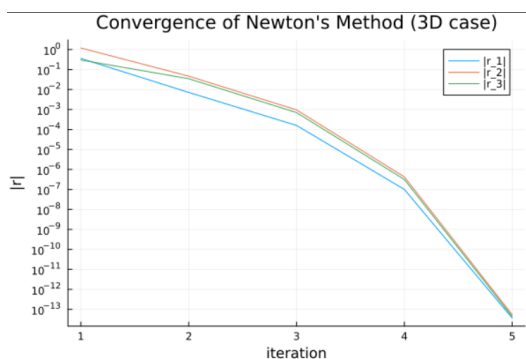
用牛顿法求多元函数的零点，迭代原理和一维牛顿法是一致的。只是 x 从一个标量

变为了一个向量。

```

1 function newtons_method(x0::Vector{
    Float64}, residual_function::
    Function; max_iters = 10)::Vector{
    Vector{Float64}}
2     X = [zeros(length(x0)) for i = 1:
    max_iters]
3     X[1] = x0
4     for i = 1:max_iters
5         # TODO: Newton's method here
6         r = residual_function(X[i])
7         if norm(r) < 1e-10
8             return X[1:i]
9         end
10        J = FD.jacobian(
    residual_function, X[i])
11        X[i+1] = X[i] - J\r
12    end
13    error("Newton did not converge")
14 end

```



1.3.3 优化中的牛顿法应用

假设有一个多变量代价函数 $f(x)$ ，目标找到他的极小值点。从微积分的角度看，极值点的梯度必然为 0，这其实也是 KKT 条件中的平稳性条件，于是便把这个优化问题转化为一个求根问题： $\nabla f(x) = \mathbf{0}$ 。

我们把残差函数设定为代价函数的梯度，即 $r(x) = \nabla f(x)$ 。然后直接套用上面的牛顿法。注意，梯度的雅可比矩阵 $\frac{\partial \nabla f(x)}{\partial x}$ ，在数学上正好就是代价函数 $f(x)$ 的海森矩阵 $\nabla^2 f(x)$ 。于是可得：

$$\Delta x = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

```

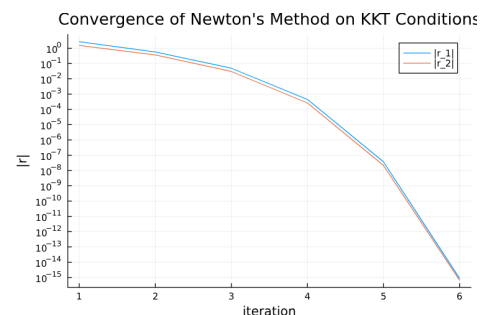
1 function kkt_conditions(x)

```

```

# TODO: return the
stationarity condition for the cost
function f ( f(x))
    return FD.gradient(f,x)
end
residual_fx(_x) = kkt_conditions(
    _x
    x0 = [-0.9512129986081451,
    0.8061342694354091]
    X = newtons_method(x0, residual_fx
    ; max_iters = 10)
    R = residual_fx.(X)
    Rp = [[abs(R[i][ii]) for i = 1:
    length(R)] for ii = 1:length(R[1])]
end

```



如果我们直接在 x_k 处对原来的代价函数 $f(x)$ 进行二阶泰勒展开：

$$f(x_k + \Delta x) \approx f(x_k) + \nabla f(x_k)^T \Delta x + \frac{1}{2} \Delta x^T [\nabla^2 f(x_k)] \Delta x$$

这是一个关于 Δx 的二次函数（开口向上的抛物面）。为了让这个二次近似函数最小化，我们对 Δx 求导并令其等于 0：

$$\nabla f(x_k) + [\nabla^2 f(x_k)] \Delta x = 0$$

解出来的结果，完美契合了刚才的牛顿步长：

$$\Delta x = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

这说明在优化问题中利用平稳性条件应用牛顿法，本质是在每次迭代中最小化代价函数近似拟合的抛物线模型。

2 HW1

2.1 Question1: 积分

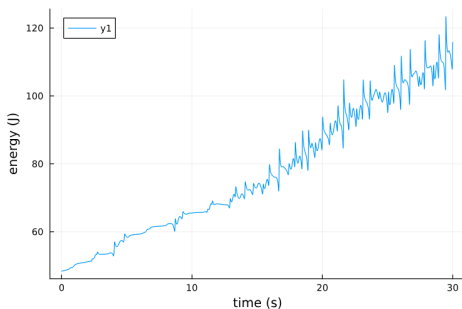
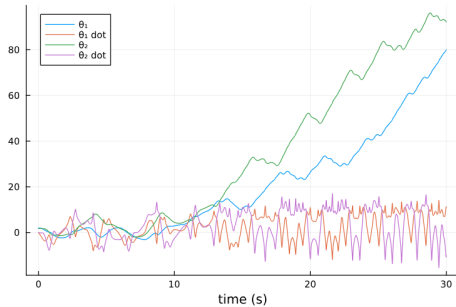
$\dot{x} = f(x)$ 。双摆的运动学方程和总能量的表示代码已经给出，本题目用不同的积分器进行模拟仿真验证结果。积分器分显式和隐式共有六种。

2.1.1 显式积分

前向欧拉法：

$$x_{k+1} = x_k + \Delta t \cdot f(x_k)$$

```
1 function forward_euler(params::
    NamedTuple, dynamics::Function, x::
    Vector, dt::Real)::Vector
2     #  $\dot{x} = \text{dynamics}(\text{params}, x)$ 
3     return x + dt*dynamics(params,x)
4 end
```



显式中点法：

$$x_m = x_k + \frac{\Delta t}{2} \cdot f(x_k)$$

$$x_{k+1} = x_k + \Delta t \cdot f(x_m)$$

```
1 function midpoint(params::NamedTuple,
    dynamics::Function, x::Vector, dt::
    Real)::Vector
2     # TODO: implement explicit
    midpoint
3     xm = x + 0.5*dt*dynamics(params,x)
4     return x + dt*dynamics(params,xm)
5 end
```

RK4 法：

$$k_1 = \Delta t \cdot f(x_k)$$

$$k_2 = \Delta t \cdot f(x_k + k_1/2)$$

$$k_3 = \Delta t \cdot f(x_k + k_2/2)$$

$$k_4 = \Delta t \cdot f(x_k + k_3)$$

$$x_{k+1} = x_k + (1/6) \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

```
1 function rk4(params::NamedTuple,
    dynamics::Function, x::Vector, dt::
    Real)::Vector
2     # TODO: implement RK4
3     k1 = dt * dynamics(params, x)
4     k2 = dt * dynamics(params, x + k1
    / 2)
5     k3 = dt * dynamics(params, x + k2
    / 2)
6     k4 = dt * dynamics(params, x + k3)
7     return x + (1 / 6) * (k1 + 2 * k2
    + 2 * k3 + k4)
8 end
```

2.1.2 隐式积分

显式积分器的工作方式是通过传入 x_k 和 Δt 作为参数来调用函数，并返回 x_{k+1}

$$x_{k+1} = f_{\text{explicit}}(x_k, \Delta t)$$

隐式积分器在状态 x_k 和 x_{k+1} 之间具有以下关系：

$$f_{\text{implicit}}(x_k, x_{k+1}, \Delta t) = 0$$

这意味着如果我们想从 x_k 得到 x_{k+1} ，我们必须求解满足上述方程的 x_{k+1} 。这是一个关于 x_{k+1} （我们的未知数）的求根问题，因此我们需要使用牛顿法。

后向欧拉法 (一阶):

$$f(x_k, x_{k+1}, \Delta t) = x_k + \Delta t \cdot \dot{x}_{k+1} - x_{k+1} = 0$$

隐式中点法 (二阶):

$$x_{k+1/2} = \frac{1}{2}(x_k + x_{k+1})$$

$$f(x_k, x_{k+1}, \Delta t) = x_k + \Delta t \cdot \dot{x}_{k+1/2} - x_{k+1} = 0$$

埃尔米特-辛普森法 (三阶):

$$x_{k+1/2} = \frac{1}{2}(x_k + x_{k+1}) + \frac{\Delta t}{8}(\dot{x}_k - \dot{x}_{k+1})$$

$$f(x_k, x_{k+1}, \Delta t) = x_k + \frac{\Delta t}{6} \cdot (\dot{x}_k + 4\dot{x}_{k+1/2} + \dot{x}_{k+1}) - x_{k+1} = 0$$

```

1 function backward_euler(params::
    NamedTuple, dynamics::Function, x1
    ::Vector, x2::Vector, dt::Real)::
    Vector
2     x1 + dt*dynamics(params,x2) - x2
3 end
4 function implicit_midpoint(params::
    NamedTuple, dynamics::Function, x1
    ::Vector, x2::Vector, dt::Real)::
    Vector
5     xm = (x1 + x2)/2
6     x1 + dt*dynamics(params,xm) - x2
7 end
8 function hermite_simpson(params::
    NamedTuple, dynamics::Function, x1
    ::Vector, x2::Vector, dt::Real)::
    Vector
9     x1 = dynamics(params, x1)
10    x2 = dynamics(params, x2)
11    xm = (1/2)*(x1 + x2) + (dt/8)*(x1
        - x2)
12    xm = dynamics(params, xm)
13    x1 + (dt/6)*(x1 + 4*xm + x2) -
        x2
14 end

```

以下是用牛顿法求解满足隐式积分方程的 x_2 。

```

1 function implicit_integrator_solve(
    params::NamedTuple, dynamics::
    Function, implicit_integrator::
    Function, x1::Vector, dt::Real;tol

```

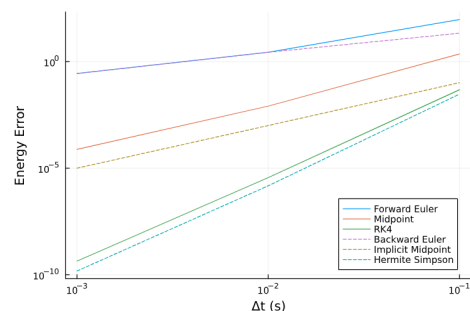
```

= 1e-13, max_iters = 10)::Vector
# initialize guess
x2 = 1*x1
# TODO: use Newton's method to
solve for x2 such that residual for
the integrator is 0

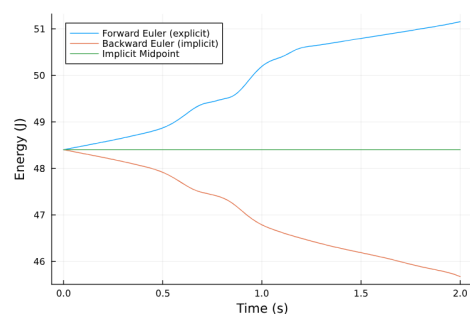
residual_fx(_x2) =
implicit_integrator(params,
dynamics, x1, _x2, dt)
for i = 1:max_iters
    residual = residual_fx(x2)
    if norm(residual)<tol
        return x2
    end
    residual_jacobian = FD.
jacobian(residual_fx, x2)
    x2 -= residual_jacobian\
residual

end
error("implicit integrator solve
failed")
end

```



通过对比能量求解表现, 发现同阶的稳定性隐式是大于显式的。另外显式欧拉法会导致不稳定, 隐式欧拉法会导致能量损失, 隐式中点法能很好的保持能量平衡。



2.2 Question2: 等式约束优化

2.2.1 牛顿法

给定问题:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{st} \quad & c(x) = 0 \end{aligned}$$

有如下拉格朗日函数:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T c(x),$$

以及如下 KKT 条件

$$\begin{aligned} \nabla_x \mathcal{L} = \nabla_x f(x) + \left[\frac{\partial c}{\partial x} \right]^T \lambda &= 0 \\ c(x) &= 0 \end{aligned}$$

本质是一个求根问题, 求解一个满足 KKT 条件的变量 $z = [x^T, \lambda]^T$

定义

$$r(z) = \begin{bmatrix} \nabla_x \mathcal{L} \\ c(x) \end{bmatrix} = \begin{bmatrix} \nabla f(x) + \left[\frac{\partial c}{\partial x} \right]^T \lambda \\ c(x) \end{bmatrix} = 0$$

计算牛顿步长为:

$$\Delta z = - \left[\frac{\partial r}{\partial z} \right]^{-1} r(z_k)$$

牛顿步 Δz 仅在局部有效。为了防止在非线性较强时步子跨得太大导致发散, 我们引入步长因子 $\alpha \in (0, 1]$:

$$z_{new} = z_{old} + \alpha \Delta z$$

同时定义一个价值函数 $\phi(z) = \|r(z)\|_2$, 只有当 $\phi(z + \alpha \Delta z) < \phi(z)$ 时, 才接受该步长。

初始设 $\alpha = 1.0$, 检查 $\phi(z + \alpha \Delta z)$ 是否减小, 若未减小, 则令 $\alpha = \frac{\alpha}{2}$ 并重复, 直到找到合适的 α 或达到最大迭代次数。这就是线搜索策略。

```

max_ls_iters = 10)

::Float64
psi0 = merit_fx(z)
= 1.0
for i = 1:max_ls_iters
    if merit_fx(z + alpha*Delta z) <
psi0
        return alpha
    else
        alpha = alpha/2
    end
end
error("linesearch failed")
end

function newtons_method(z0::Vector,
    res_fx::Function, res_jac_fx::
Function, merit_fx::Function;
    tol = 1e-10,
    max_iters = 50, verbose = false)::
Vector{Vector{Float64}}
    for i = 1:(max_iters - 1)
        r = res_fx(Z[i])
        norm_r = norm(r)
        if verbose
            print("iter: $i      |r|:
$norm_r      ")
        end

        # check convergence with norm
of residual < tol
        if norm_r < tol
            return Z[1:i]
        end

        # caculate Newton step (don't
forget the negative sign)
        J = res_jac_fx(Z[i])
        Delta z = -J\r

        # linesearch and update z
        alpha = linesearch(Z[i], Delta z,
merit_fx)
        Z[i+1] = Z[i] + alpha*Delta z
        if verbose
            print("alpha: $alpha \n")
        end
    end
    error("Newton's method did not
converge")
end

```

```

1 function linesearch(z::Vector, Delta z::
Vector, merit_fx::Function;

```

2.2.2 全牛顿法和高斯牛顿法

$$r(z) = \begin{bmatrix} \nabla f(x) + A(x)^T \lambda \\ c(x) \end{bmatrix}$$

$$\frac{\partial r}{\partial z} = \begin{bmatrix} H & A(x)^T \\ A(x) & -\rho I \end{bmatrix}$$

$A(x) = \frac{\partial c}{\partial x}$ 是约束函数的雅可比矩阵

ρ 是正则化系数 $1e-3$

在全牛顿法中, H 是对整个拉格朗日函数 $\mathcal{L}(x, \lambda)$ 求二阶导。不仅考虑了目标函数的曲率 $\nabla^2 f(x)$, 还考虑了约束曲面的曲率 $\lambda \nabla^2 c(x)$:

$$H_{FN} = \nabla_{xx}^2 \mathcal{L}(x, \lambda) = \nabla^2 f(x) + \lambda \nabla^2 c(x)$$

在高斯牛顿法中忽略了第二项:

$$H_{GN} \approx \nabla^2 f(x)$$

```

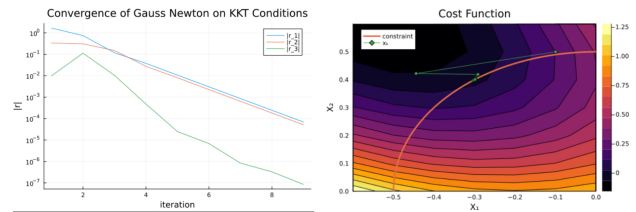
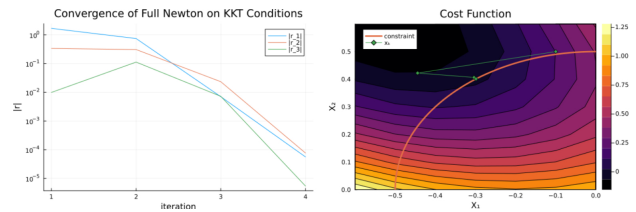
1 function function kkt_conditions(z::
  Vector)::Vector
2
3     x = z[1:2]
4     lambd = z[3:3]
5
6     A = constraint_jacobian(x)
7     return [
8         FD.gradient(cost, x) +
9         transpose(A)*lambd;
10        constraint(x)
11    ]
12 end
13
14 function fn_kkt_jac(z::Vector)::Matrix
15
16     x = z[1:2]
17     lambd = z[3]
18
19     _lag(_x) = cost(_x) + lambd *
20     constraint(_x)
21     transpose(A) = FD.gradient(
22     constraint, x)
23     H = FD.hessian(_lag, x)
24     rho = 1e-3
25     return [H+rho*I transpose(A); A -
26     rho*I]
27 end

```

```

25 function gn_kkt_jac(z::Vector)::Matrix
26
27     x = z[1:2]
28     lambd = z[3]
29
30     transpose(A) = FD.gradient(
31     constraint, x)
32     H=FD.hessian(cost, x)
33     rho = 1e-3
34     return [H+rho*I transpose(A); A -
35     rho*I]
36 end

```



2.2.3 平衡四足机器人

求解控制输入 $u \in \mathbb{R}^{12}$ 和状态 $x \in \mathbb{R}^{30}$, 使得四足机器人能以单腿支撑在平衡点上。首先从 quadruped.jl 加载动力学模型: $\dot{x} = f(x, u) = \text{dynamics}(\text{model}, x, u)$

待解决的优化问题为:

$$\begin{aligned} \min_{x, u} \quad & \frac{1}{2}(x - x_{\text{guess}})^T(x - x_{\text{guess}}) + \frac{1}{2}10^{-3}u^T u \\ \text{st} \quad & \dot{x} = f(x, u) = 0 \end{aligned}$$

其中:

原始变量: $x \in \mathbb{R}^{30}$ (状态) 和 $u \in \mathbb{R}^{12}$ (控制), 堆叠为 $y = [x^T, u^T]^T \in \mathbb{R}^{42}$ 。

约束条件: $\dot{x} = 0$, 确保结果配置处于平衡状态。

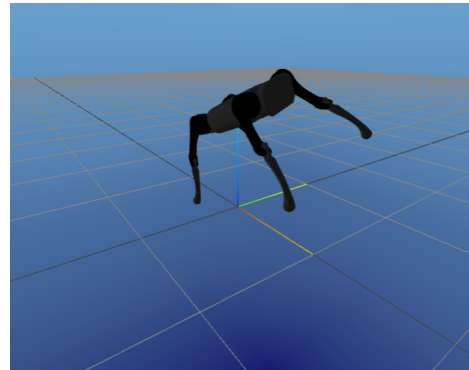
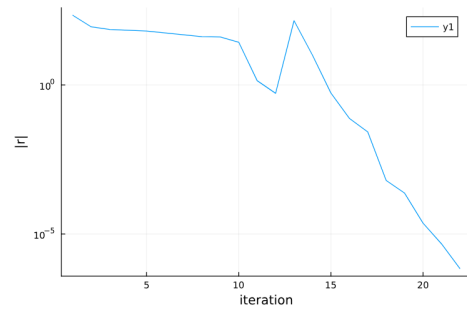
对偶变量: $\lambda \in \mathbb{R}^{30}$ 。

求解变量: $z = [y^T, \lambda^T]^T \in \mathbb{R}^{72}$ 。

```

1  const idx_x = 1:30
2  const idx_u = 31:42
3  const idx_c = 43:72
4  function quadruped_cost(y::Vector)
5      # cost function
6      @assert length(y) == 42
7      x = y[idx_x]
8      u = y[idx_u]
9      # TODO: return cost
10     return 0.5*norm(x - x_guess)^2 +
        0.5*1e-3*norm(u)^2
11 end
12 function quadruped_constraint(y::
    Vector)::Vector
13     # constraint function
14     @assert length(y) == 42
15     x = y[idx_x]
16     u = y[idx_u]
17     # TODO: return constraint
18     return dynamics(model, x, u)
19 end
20 function quadruped_kkt(z::Vector)::
    Vector
21     @assert length(z) == 72
22     x = z[idx_x]
23     u = z[idx_u]
24     lambd = z[idx_c]
25     y = [x;u]
26     # TODO: return the KKT conditions
27     return [
28 FD.gradient(quadruped_cost,y) +
        transpose(FD.jacobian(
            quadruped_constraint, y))*lambd;
29         quadruped_constraint(y)
30     ]
31 end
32 function quadruped_kkt_jac(z::Vector)
    ::Matrix
33     @assert length(z) == 72
34     x = z[idx_x]
35     u = z[idx_u]
36     lambd = z[idx_c]
37     y = [x;u]
38     # TODO: return Gauss-Newton
    Jacobian with a regularizer
39     A = FD.jacobian(
        quadruped_constraint, y)
40     H = FD.hessian(quadruped_cost, y)
41     rho = 1e-4
42     return [(H + rho*I) transpose(A);
        A -rho*I]
43 end

```



1、为什么要使用线搜索?

线搜索通过动态缩小步长来确保每次迭代都能严格降低价值函数，从而防止牛顿法在强非线性区域因步子迈得太大而导致计算发散。

2、对于凸问题和非凸问题，我们都需要线搜索吗?

是的，即便目标函数是凸的，如果初始点距离最优解较远，不带线搜索的纯牛顿法依然可能因为超调而产生振荡甚至发散

3、举出一个绝对不需要线搜索的情况。

当目标函数是二次函数且约束条件为线性约束时(即标准的二次规划问题 QP)，由于拉格朗日函数的一阶泰勒展开完全精确，牛顿法只需迈出完整的整步 ($\alpha = 1$) 即可一步精确收敛。

2.3 Question3: 对数域内点二次规划 (QP) 求解器

$$\begin{aligned}
 \min_x \quad & \frac{1}{2}x^T Qx + q^T x \\
 \text{s.t.} \quad & Ax - b = 0 \\
 & Gx - h \geq 0
 \end{aligned}$$

2.3.1 KKT 条件

引入用于等式约束的拉格朗日乘子 μ ，以及用于不等式约束的拉格朗日乘子 λ ，并填写该 QP 问题的 KKT 条件。对于互补松弛条件，用 $\circ$ 符号

$$\begin{aligned} Qx + q + A^T \mu - G^T \lambda &= 0 & (\text{定常条件}) \\ Ax - b &= 0 & (\text{原始可行性}) \\ Gx - h &\geq 0 & (\text{原始可行性}) \\ \lambda &\geq 0 & (\text{对偶可行性}) \\ \lambda \circ (Gx - h) &= 0 & (\text{互补松弛}) \end{aligned}$$

2.3.2 互补松弛条件

为了应用对数域技巧，我们可以引入一个松弛变量 (Slack Variable) s 来表示不等式约束。这个新变量允许我们通过使用“对数域替换”（其构造结果始终为正）来强制满足不等式约束 ($s \geq 0$)。

通过以下步骤修改你的 KKT 条件：

- 1、添加松弛变量 s ，将原始可行性中的 $Gx - h \geq 0$ 拆分为 $Gx - h = s$ 且 $s \geq 0$ 。
- 2、松弛互补条件，使 $\lambda \circ s = 0$ 变为 $\lambda \circ s = 1\rho$ ，其中 ρ 是正的障碍参数 (Barrier Parameter)， $\mathbf{1}$ 是全 1 向量。

$$\begin{aligned} Qx + q + A^T \mu - G^T \lambda &= 0 \\ Ax - b &= 0 \\ Gx - h - s &= 0 \\ s &\geq 0 \\ \lambda &\geq 0 \\ \lambda \circ s &= 1\rho \end{aligned}$$

2.3.3 对数域变量代换

为了对 λ 和 s 强加正定约束，我们可以进行变量代换。通过采用特定的代换形式 $\lambda = \sqrt{\rho}e^{-\sigma}$ 和 $s = \sqrt{\rho}e^{\sigma}$ ，我们可以直接确保松弛互补条件 $\lambda \circ s = 1\rho$ 恒成立。

完成以下步骤：

- 1、定义新变量 σ ，并令 $\lambda = \sqrt{\rho}e^{-\sigma}$ 且 $s = \sqrt{\rho}e^{\sigma}$ 。

2、将 KKT 条件中的 λ 和 s 替换为上述新定义。

三个 KKT 条件由于这种构造方式已经默认自动满足，另外三个如下：

$$\begin{aligned} Qx + q + A^T \mu - G^T \sqrt{\rho}e^{-\sigma} &= 0 \\ Ax - b &= 0 \\ Gx - h - \sqrt{\rho}e^{\sigma} &= 0 \end{aligned}$$

2.3.4 对数域求解器

```

1 function kkt_conditions(qp::NamedTuple
2     , z::Vector, rho::Float64)::Vector
3     x, mu, sigma = z[qp.xi], z[qp.mui
4     ], z[qp.sigmai]
5     lambd = sqrt(rho) .* exp.(-sigma)
6     return [
7         qp.Q*x + qp.q + qp.A'*mu - qp.
8         G'*lambd
9         c_eq(qp, x)
10        min.(h_ineq(qp, x), 0)
11        min.(lambd, 0)
12        lambd .* h_ineq(qp, x)
13    ]
14 end
15
16 function ip_kkt_conditions(qp::
17     NamedTuple, z::Vector, rho::Float64
18     )::Vector
19     x, mu, sigma = z[qp.xi], z[qp.mui
20     ], z[qp.sigmai]
21     s, lambd = sqrt(rho) .* exp.(sigma)
22     , sqrt(rho) .* exp.(-sigma)
23     return [
24         qp.Q*x + qp.q + qp.A'*mu - qp.
25         G'*lambd
26         c_eq(qp, x)
27         h_ineq(qp, x) - s
28     ]
29 end
30
31 function ip_kkt_jac(qp::NamedTuple, z
32     ::Vector, rho::Float64)::Matrix
33     x, mu, sigma = z[qp.xi], z[qp.mui
34     ], z[qp.sigmai]
35     return [
36         qp.Q qp.A' qp.G'*diag(sqrt(
37         rho).*exp.(-sigma))
38         qp.A zeros(size(qp.A, 1), size
39         (qp.A, 1) + size(qp.G, 1))

```

```

28     qp.G zeros(size(qp.G, 1), size
    (qp.A, 1)) -diagm(sqrt(rho).*exp.(
    sigma))
29 ]
30 end
31 # 日志打印函数
32 function logging(qp::NamedTuple,
    main_iter::Int, z::Vector, rho::
    Real, alpha::Real)
33     x, mu, sigma = z[qp.xi], z[qp.mui
    ], z[qp.sigmai]
34     lambd = sqrt(rho) .* exp.(-sigma)
35     stationarity_norm = norm(qp.Q*x +
    qp.q + qp.A'*mu - qp.G'*lambd)
36
37     @printf("%3d % 7.2e % 7.2e %
    7.2e % 7.2e %5.0e %5.0e\n",
38         main_iter, stationarity_norm
    , minimum(h_ineq(qp,x)),
39         norm(c_eq(qp,x),Inf), abs(
    dot(lambd,h_ineq(qp,x))), rho,
    alpha)
40 end
41
42 function solve_qp(qp; verbose = true,
    max_iters = 100, tol = 1e-8)
43     # 初始化解向量
44     z = zeros(length(qp.q) + length(qp
    .b) + length(qp.h))
45
46     rho = 0.1
47     for main_iter = 1:max_iters
48         # 计算当前内点法残差和雅可比
49         ip_res = ip_kkt_conditions(qp,
    z, rho)
50         ip_jac = ip_kkt_jac(qp, z, rho
    )
51         # 求解牛顿步长方向
52         Δz = -ip_jac \ ip_res
53         # 回溯线搜索 (以内点法残差范数
    作为价值函数)
54         alpha = 1.0
55         for ls_iters = 1:10
56             if norm(ip_kkt_conditions(
    qp, z + alpha*Δz, rho)) < norm(
    ip_res)
57                 break
58             end
59             alpha = alpha / 2
60         end
61         # 更新状态
62         z = z + alpha * Δz
63

```

```

64         if verbose
65             logging(qp, main_iter, z,
    rho, alpha)
66         end
67         # 收敛性与屏障参数更新判别
68         if norm(kkt_conditions(qp, z,
    rho), Inf) < tol
69             x, mu, lambd = z[qp.xi], z
    [qp.mui], sqrt(rho) .* exp.(-z[qp.
    sigmai])
70             return x, mu, lambd
71         elseif norm(ip_kkt_conditions(
    qp, z, rho), Inf) < tol
72             rho = rho * 0.1
73         end
74     end
75     error("qp solver did not converge"
    )
76 end

```

2.3.5 用 QP 模拟下落的砖块

下落的动力学方程可以写作：

$$M\dot{v} + Mg = J^T \mu$$

其中： $M = mI_{2 \times 2}$, $g = \begin{bmatrix} 0 \\ 9.81 \end{bmatrix}$ 。

$J = \begin{bmatrix} 0 & 1 \end{bmatrix}$, $\mu \in \mathbb{R}$ 是接触面法向力。

速度 $v \in \mathbb{R}^2$ 和位置 $q \in \mathbb{R}^2$ 分别由水平和垂直分量构成。

可以使用后向欧拉法 (Backward Euler) 对动力学进行离散化：

$$\begin{bmatrix} v_{k+1} \\ q_{k+1} \end{bmatrix} = \begin{bmatrix} v_k \\ q_k \end{bmatrix} + \Delta t \cdot \begin{bmatrix} \frac{1}{m} J^T \mu_{k+1} - g \\ v_{k+1} \end{bmatrix}$$

同时，系统还受到以下接触约束

$$Jq_{k+1} \geq 0 \quad (\text{不要穿透冰面})$$

$$\mu_{k+1} \geq 0 \quad (\text{法向力只能推，不能拉})$$

$$\mu_{k+1} Jq_{k+1} = 0 \quad (\text{非接触时没有非接触力 / 互补松弛})$$

2.3.6 下落砖块的 QP 问题表述

minimize $v_{k+1} \quad \frac{1}{2} v_{k+1}^T M v_{k+1} + [M(\Delta t \cdot g - v_k)]^T v_{k+1}$
 subject to $-J(q_k + \Delta t \cdot v_{k+1}) \leq 0$

该问题的 KKT 条件，并证明离散动力学方程与本 QP 问题完全等价。

拉格朗日函数：

$$\mathcal{L}(v_{k+1}, \mu_{k+1}) = \frac{1}{2} v_{k+1}^T M v_{k+1} + [M(\Delta t \cdot g - v_k)]^T v_{k+1} + \mu_{k+1} [-J(q_k + \Delta t \cdot v_{k+1})]$$

定常条件：

$$\begin{aligned} \nabla_{v_{k+1}} \mathcal{L} &= M v_{k+1} + M(\Delta t \cdot g - v_k) - \Delta t \cdot J^T \mu_{k+1} = 0 \\ M v_{k+1} &= M v_k - \Delta t \cdot M g + \Delta t \cdot J^T \mu_{k+1} \end{aligned}$$

原始可行性：

$$-J(q_k + \Delta t \cdot v_{k+1}) \leq 0 \implies J(q_k + \Delta t \cdot v_{k+1}) \geq 0$$

对偶可行性：

$$\mu_{k+1} \geq 0$$

互补松弛性：

$$\mu_{k+1} \cdot [J(q_k + \Delta t \cdot v_{k+1})] = 0$$

分析先前由后向欧拉法得到的离散化动力学模型：

根据位置离散方程有： $q_{k+1} = q_k + \Delta t \cdot v_{k+1}$ 。将其代入速度离散方程 $v_{k+1} = v_k + \Delta t \cdot (\frac{1}{m} J^T \mu_{k+1} - g)$ 中，并在等式两边同时乘以质量矩阵 $M = mI$ ，移项后便得到了定常条件： $M v_{k+1} = M v_k + \Delta t \cdot J^T \mu_{k+1} - \Delta t \cdot M g$ 将 $q_{k+1} = q_k + \Delta t \cdot v_{k+1}$ 代入原本三个接触物理约束：

不穿透冰面： $J q_{k+1} \geq 0 \implies J(q_k + \Delta t \cdot v_{k+1}) \geq 0$ ，完全对应 QP 的原始可行性

法向力只推不拉： $\mu_{k+1} \geq 0$ ，完全对应 QP 的对偶可行性

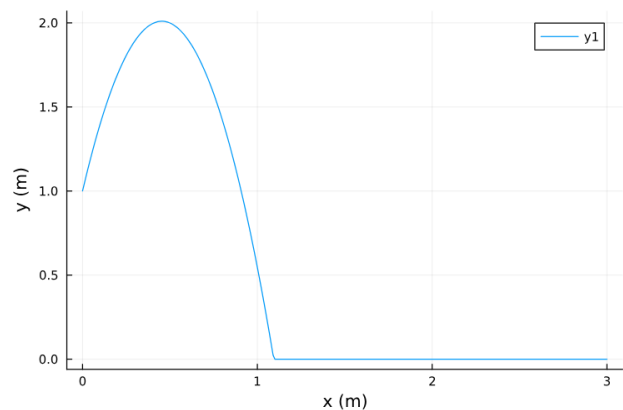
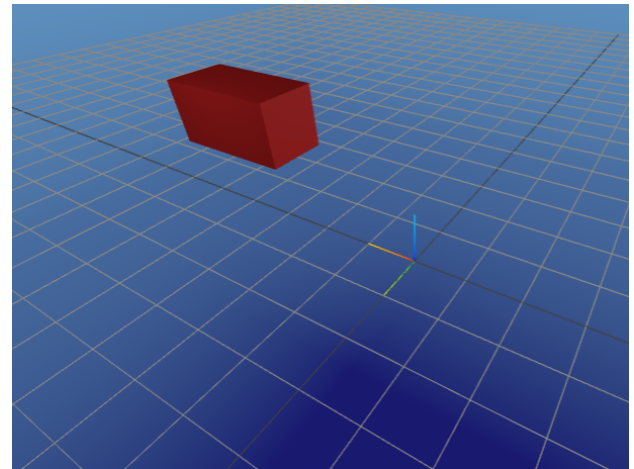
非接触无作用力： $\mu_{k+1} J q_{k+1} = 0 \implies \mu_{k+1} J(q_k + \Delta t \cdot v_{k+1}) = 0$ ，完全对应 QP 的互补松弛性

因此优化问题的 KKT 条件与砖块下落滑行的离散时间物理动力学方程及接触约束具有完全相同的数学表达，拉格朗日乘子 μ_{k+1} 的物理意义即为地面对砖块的法向接触

力，求解该 QP 问题在数学上与求解一个时步的物理动力学仿真完全等价。

2.3.7 砖块模拟

```
function brick_simulation_qp(q, v;
    mass = 1.0, Δt = 0.01)
    qp = (
        Q = [mass 0; 0 mass],
        q = [mass 0; 0 mass]*(Δt*[0;
9.81] - v),
        A = zeros(0,2),
        b = zeros(0),
        G = [0 Δt],
        h = [0 -1] * q,
        xi = 1:2,
        mui = [],
        sigmai = 3:3
    )
    return qp
end
```



2.3.8 求解 QP 问题

$$\begin{aligned} \min_{y \in \mathbb{R}^2, a \in \mathbb{R}, b \in \mathbb{R}} \quad & \frac{1}{2} y^T \begin{bmatrix} 1 & .3 \\ .3 & 1 \end{bmatrix} y + a^2 + 2b^2 \\ & + [-2 \quad 3.4] y + 2a + 4b \\ \text{st} \quad & a + b = 1 \\ & [-1 \quad 2.3] y + a - 2b = 3 \\ & -0.5 \leq y \leq 1 \\ & -1 \leq a \leq 1 \\ & -1 \leq b \leq 1 \end{aligned}$$

```

1  Q = [
2      1.0    .3    0    0;
3      0.3    1.0    0    0;
4      0      0    2    0;
5      0      0    0    4
6  ]
7  q = [-2, 3.4, 2, 4]
8  A = [
9      0 0 1 1;
10     -1 2.3 1 -2
11  ]
12  b = [1, 3]
13  G = [-diagm(ones(4)); diagm(ones
14      (4))]
15  primal_max = [1, 1, 1, 1.]
16  primal_min = [-.5, -.5, -1, -1]
17  h = [-primal_max; primal_min]
18  qp = (Q=Q, q=q, A=A, b=b, G=G, h=h
19      , xi=1:4, i=5:6, i=7:14)
20  x, _, _ = solve_qp(qp; verbose = true,
21      tol = 1e-6)
22  y = x[1:2]
23  a = x[3]
24  b = x[4]

```

```

7  1.08e-12  1.70e-02  0.00e+00  8.00e-02  1e-01  1e+00
8  1.32e+00  6.45e-03  0.00e+00  1.02e-01  1e-02  5e-01
9  1.68e-01  1.67e-03  0.00e+00  7.94e-02  1e-02  1e+00
10 2.40e-03  1.79e-03  0.00e+00  8.00e-02  1e-02  1e+00
11 5.12e-07  1.79e-03  0.00e+00  8.00e-02  1e-02  1e+00
12 4.82e-01  5.85e-04  0.00e+00  1.04e-02  1e-03  5e-01
13 2.22e-02  1.80e-04  0.00e+00  7.98e-03  1e-03  1e+00
14 4.44e-05  1.82e-04  4.44e-16  8.00e-03  1e-03  1e+00
15 1.79e-10  1.82e-04  0.00e+00  8.00e-03  1e-03  1e+00
16 3.83e-01  5.77e-05  0.00e+00  1.05e-03  1e-04  5e-01
17 1.46e-02  1.81e-05  4.44e-16  7.98e-04  1e-04  1e+00
18 1.94e-05  1.82e-05  0.00e+00  8.00e-04  1e-04  1e+00
19 3.41e-11  1.82e-05  0.00e+00  8.00e-04  1e-04  1e+00
20 3.73e-01  5.76e-06  4.44e-16  1.05e-04  1e-05  5e-01
21 1.39e-02  1.81e-06  0.00e+00  7.98e-05  1e-05  1e+00
22 1.76e-05  1.82e-06  4.44e-16  8.00e-05  1e-05  1e+00
23 2.82e-11  1.82e-06  0.00e+00  8.00e-05  1e-05  1e+00
...
26 1.74e-05  1.82e-07  0.00e+00  8.00e-06  1e-06  1e+00
27 2.77e-11  1.82e-07  4.44e-16  8.00e-06  1e-06  1e+00

```

3 HW2

3.1 Question1: 有限时间步 LQR

考虑一个“双积分器 (double integrator)”动力学模型。该系统的状态为 $x \in \mathbb{R}^4$ ，控制输入为 $u \in \mathbb{R}^2$ 。其中状态描述了物体在二维空间中的位置 p 和速度 v ，控制输入则是该物体的加速度 a 。状态与控制定义如下：

$$x = [p_1, p_2, v_1, v_2]$$

$$u = [a_1, a_2]$$

该系统的连续时间动力学方程如下：

$$\dot{x} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} u$$

3.1.1 模型离散化

利用矩阵指数 (Julia 中的 `exp` 函数)，在假设控制输入为零阶保持器 (Zero-Order Hold, ZOH) 的前提下，对连续时间模型进行离散化。

```

1 function double_integrator_AB(dt)::
2     Tuple{Matrix, Matrix}
3     Ac = [0 0 1 0;
4           0 0 0 1;
5           0 0 0 0;
6           0 0 0 0.]
7     Bc = [0 0;
8           0 0;
9           1 0;
10          0 1]
11     nx, nu = size(Bc)
12
13     H = exp([Ac Bc; zeros(nu, nx + nu)] * dt)
14     A = H[1:nx, 1:nx]
15     B = H[1:nx, (nx + 1):(nx + nu)]
16
17     return A, B
18 end

```

3.1.2 基于凸优化的有限时间步 LQR

利用凸优化来求解有限时间步的 LQR 问题。该问题要求状态权重矩阵 $Q \in S_+$ (对

称半正定), 控制权重矩阵 $R \in S_{++}$ (对称正定)。据此, 优化问题可以表示为:

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \sum_{i=1}^{N-1} \left[\frac{1}{2} x_i^T Q x_i + \frac{1}{2} u_i^T R u_i \right] + \frac{1}{2} x_N^T Q_f x_N \\ \text{st} \quad & x_1 = x_{IC} \\ & x_{i+1} = A x_i + B u_i \\ & \text{for } i = 1, \dots, N-1 \end{aligned}$$

由于代价函数 (Cost function) 是凸二次函数, 且约束条件均为线性等式约束, 因此这是一个凸优化问题。

```

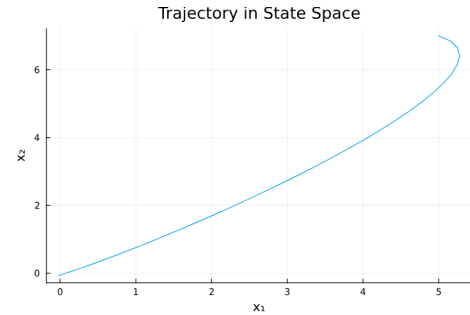
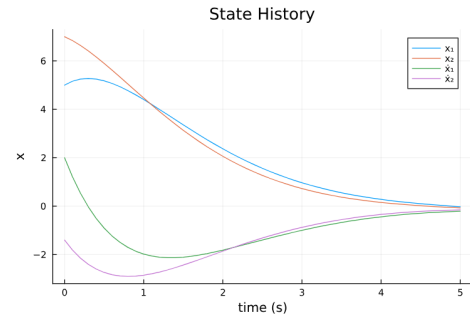
1  X = cvx.Variable(nx, N)
2  U = cvx.Variable(nu, N - 1)
3  # create cost
4  cost = 0
5  for k = 1:(N-1)
6      xk = X[:,k]
7      uk = U[:,k]
8      cost += 0.5*cvx.quadform(xk,Q)
9      + 0.5*cvx.quadform(uk,R)
10 end
11 xn = X[:,N]
12 cost += 0.5*cvx.quadform(xn, Qf)
13 # initialize cvx problem
14 prob = cvx.minimize(cost)
15 # TODO: initial condition
16 constraint
17 # initial condition constraint
18 prob.constraints = vcat(prob.
19 constraints, X[:,1] == x_ic)
20 for k = 1:(N-1)
21     # dynamics constraints
22     prob.constraints= vcat(prob.
23 constraints, (X[:,k+1] == A*X[:,k]
24 + B*U[:,k]))
25 end

```

```

1 # TODO dynamics simulation
2 Xsim = [zeros(nx) for i = 1:N]
3 Xsim[1] = 1*x_ic
4 for i = 1:N-1
5     Xsim[i+1] = A*Xsim[i] + B*Ucvx
6     [i]
7 end

```



3.1.3 基于 Riccati 递归的有限时间步 LQR

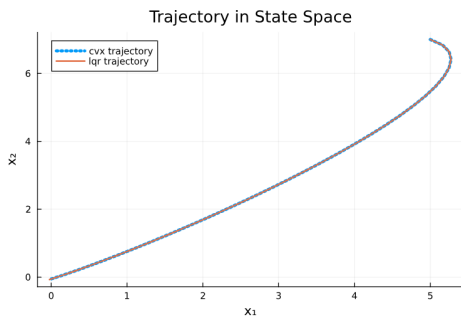
通过 Riccati 倒向递归 来求解相同的有限时间步 LQR 问题。我们将最优剩余代价函数定义为:

$$V_k(x) = \frac{1}{2} x^T P_k x$$

```

1 # check sizes of everything
2 nx, nu = size(B)
3 @assert size(A) == (nx, nx)
4 @assert size(Q) == (nx, nx)
5 @assert size(R) == (nu, nu)
6 @assert size(Qf) == (nx, nx)
7 # instantiate S and K
8 P = [zeros(nx,nx) for i = 1:N]
9 K = [zeros(nu,nx) for i = 1:N-1]
10 # initialize S[N] with Qf
11 P[N] = deepcopy(Qf)
12 # Ricatti TODO:recursion
13 for k = (N-1):(-1):1
14     K[k] = (R + B'*P[k+1]*B)\(B'*P
15 [k+1]*A)
16     P[k] = Q + K[k]'*R*K[k] + (A -
17 B*K[k])'P[k+1]*(A - B*K[k])
18 end
19 return P, K

```

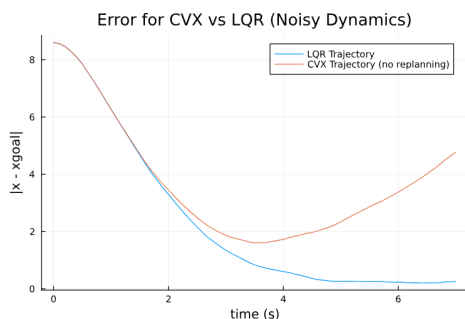
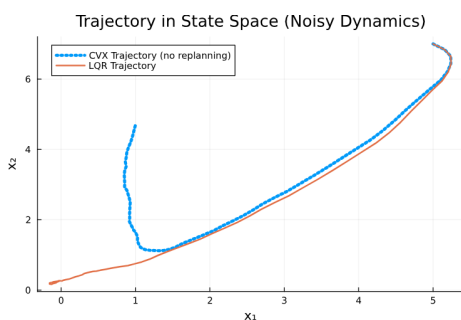


3.1.4 LQR 的两个优势

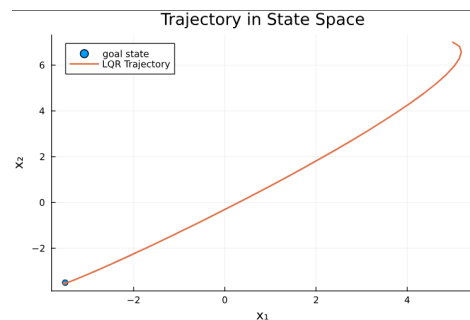
1、对噪声和模型不确定性具有强鲁棒性 (如果是纯开环的凸优化方法, 一旦实际状态偏离预期, 就必须在每个时间步重新求解优化问题, 即 MPC 思想)。

2、可以通过修改控制律 $u = -K(x - x_{\text{goal}})$, 将系统驱动至任意可达的目标状态。

测试一: 在动力学系统中加入了高斯白噪声。仿真结果表明, 在不重新规划的情况下, 开环凸优化轨迹很快被噪声带偏, 而 LQR 反馈闭环可以完美抵抗噪声, 使系统稳定。

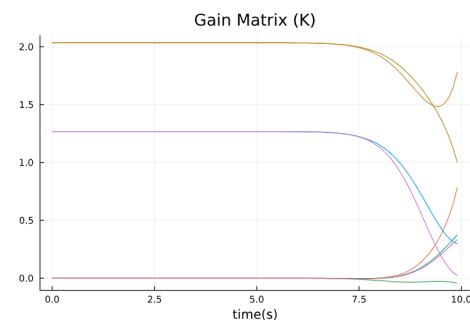
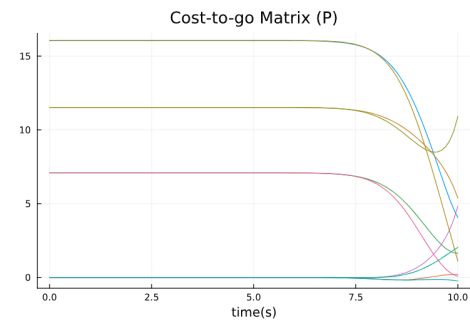


测试二: 定了一个非零的目标状态 $x_{\text{goal}} = [-3.5, -3.5, 0, 0]$ (目标位置为 -3.5, 目标速度为 0)。通过平移状态产生的反馈控制指令成功把物体驱动到了指定目标点。



3.1.5 无限时间步 LQR

在有限时间步中, 每个步长都有一个不同的反馈增益矩阵 K_k 。然而, 随着轨迹长度 (时域) 趋于无穷大 ($N \rightarrow \infty$), 初始阶段的反馈矩阵 K_1 将会收敛到一个常数矩阵, 也就是无限时间步 LQR 增益。如图所示:



3.2 Question2: 小车倒立摆模型的 LQR

接下来将研究 LQR 在非线性小车倒立摆系统中的两种不同应用。该系统包含小车的位置 p 和摆杆的仰角 θ 。

首先将该系统的非线性离散动力学在 $p = 0$ 且 $\theta = 0$ (无初始速度) 的点附近进行线性化。然后将围绕这个线性化后的状态, 使用一个无限时间步 LQR 控制器来让倒立摆在目标状态稳定下来。

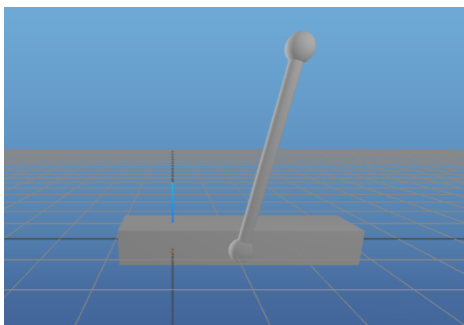
3.2.1 围绕平衡点的无限时间步 LQR

将求解无限时间步的 LQR 增益，并利用它使小车倒立摆在不稳定的平衡点（即摆杆垂直向上 $x = [0, \pi, 0, 0]$ ）保持稳定。

```

1  # states and control sizes
2  nx = 4
3  nu = 1
4  # desired x and g
5  xgoal = [0, pi, 0, 0]
6  ugoal = [0]
7  # initial condition
8  x0 = [0, pi, 0, 0] + [1.5, deg2rad
    (-20), .3, 0]
9  # simulation size
10 dt = 0.1
11 tf = 5.0
12 t_vec = 0:dt:tf
13 N = length(t_vec)
14 X = [zeros(nx) for i = 1:N]
15 X[1] = x0
16 # estimated parameters
17 params_est = (mc = 1.0, mp = 0.2, l =
    0.5)
18 # real paremeters
19 params_real = (mc = 1.2, mp = 0.16, l =
    0.55)
20 # cost terms
21 Q = diagm([1,1,.05,.1])
22 R = 0.1*diagm(ones(nu))
23 A = FD.jacobian(_x->rk4(params_est, _x
    , ugoal, dt), xgoal)
24 B = FD.jacobian(_u->rk4(params_est,
    xgoal, _u, dt), ugoal)
25 Sinf, Kinf = ihlqr(A,B,Q,R)
26 for i = 1:(N-1)
27     u = -Kinf*(X[i] - xgoal)
28     X[i+1] = rk4(params_real, X[i], u,
        dt)
29 end

```



3.2.2 无限时间步 LQR 的吸引域

我们已经为倒立摆构建了一个基于线性化动力学模型的控制器。该线性化是在目标点 (x_{goal}, u_{goal}) 处展开的，因此我们只能期望该模型在靠近线性化点时才是准确的（类似于小角度近似）。

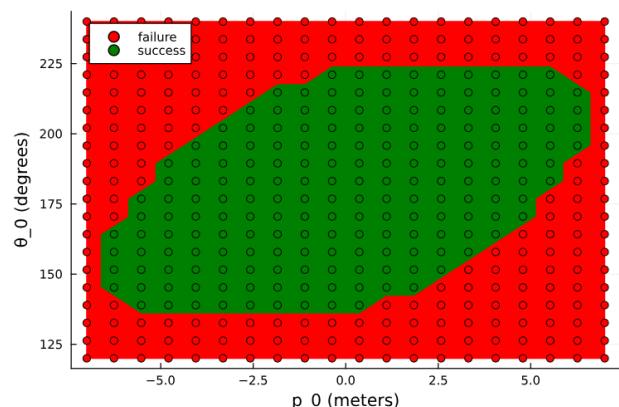
当我们远离目标状态时，线性化模型会变得越来越不准确，这会导致控制器的性能急剧下降。在达到某一个临界点后，由于这种“模型失配 (Model Mismatch)”，控制器将彻底无法稳定倒立摆。

为了证明这一点，在一系列不同的初始条件下进行测试，并且由控制器是否稳定住了倒立摆标记这些点，从而可视化出系统的吸引域，即状态空间中能够确保控制器稳定系统的区域。

```

1  A = FD.jacobian(_x->rk4(params_est
    , _x, ugoal, dt), xgoal)
2  B = FD.jacobian(_u->rk4(params_est
    , xgoal, _u, dt), ugoal)
3
4  Q = diagm([1,1,.01,.01])
5  R = 1*diagm(ones(nu))
6
7  Sinf, Kinf = ihlqr(A,B,Q,R)
8  U = [[0.0] for i = 1:N-1] #zeros(N
    -1)
9
10 for i = 1:(N-1)
11     U[i] = clamp.(-Kinf*(X[i] -
        xgoal), -3.0, 3.0)
12     X[i+1] = rk4(params_real, X[i]
        ], U[i], dt)
13 end

```



3.2.3 无限时间步 LQR 的代价函数调参

调节 LQR 的代价函数（权重矩阵），以满足以下性能指标要求：

$$\|x(5.0) - x_{\text{goal}}\|_2 < 0.1$$

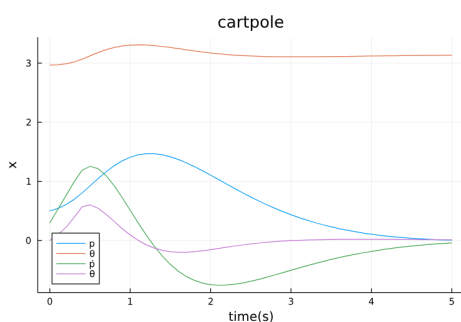
这意味着在第 5 秒时，状态与目标的 L_2 范数必须小于 0.1。

与此同时还必须应对执行器输出限制： $-3 \leq u \leq 3$ 。由于标准的 LQR 控制器无法直接把硬性输入约束考虑在内，我们需要通过精细调节代价函数，来避免控制量在执行器饱和边界上停留太久。

```

1  A = FD.jacobian(_x->rk4(params_est
2  , _x, ugoal, dt), xgoal)
3  B = FD.jacobian(_u->rk4(params_est
4  , xgoal, _u, dt), ugoal)
5
6  Q = diagm([1,1,.01,.01])
7  R = 1*diagm(ones(nu))
8
9  Sinf, Kinf = ihlqr(A,B,Q,R)
10 U = [[0.0] for i = 1:N-1] #zeros(N
11 -1)
12
13 for i = 1:(N-1)
14     U[i] = clamp.(-Kinf*(X[i] -
15 xgoal), -3.0, 3.0)
16     X[i+1] = rk4(params_real, X[i
17 ], U[i], dt)
18 end

```



3.2.4 用于轨迹跟踪的时变 LQR (TVLQR)

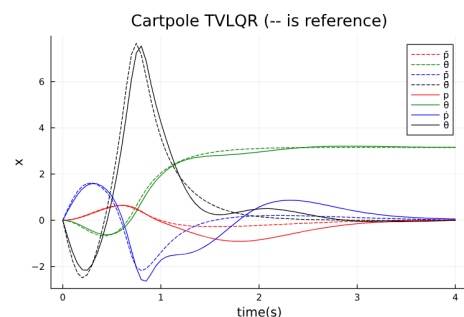
已知一条针对估计参数优化好的起摆轨迹，如果直接把这套控制输入应用到真实参数，起摆肯定会失败。

为了填补这一“仿真与现实的差距”，我们将使用一个时变 LQR (Time-Varying LQR, TVLQR) 控制器来对该参考轨迹进行精确的动态跟踪。

```

1  # cost terms
2  Q = diagm([1,1,.05,.1])
3  Qf = 10*Q
4  R = 0.05*diagm(ones(nu))
5
6  As = [FD.jacobian(_x->rk4(
7  params_est, _x, Ubar[i], dt), Xbar[
8  i]) for i=1:(N-1)]
9  Bs = [FD.jacobian(_u->rk4(
10 params_est, Xbar[i], _u, dt), Ubar[
11 i]) for i=1:(N-1)]
12
13 S,K = tvlqr(As,Bs,Q,R,Qf)
14
15 for i = 1:(N-1)
16     Δx = X[i] - Xbar[i]
17     Δu = -K[i]*Δx
18     u = Ubar[i] + Δu
19     X[i+1] = rk4(params_real, X[i
20 ], u, dt)
21 end

```



3.3 Question3: 最优交会与对接

LQR 并不是万能的，它仅仅是凸轨迹优化问题中非常小的一个子集。控制问题通常带有约束条件，这是标准 LQR 无法直接处理的，即便加上这些约束后轨迹优化问题依然是凸问题且易于求解，但在这些情况下，我们无法再通过解出一个简易的优化反馈增益 K 来应用闭环反馈策略。

解决这个问题的方案就是模型预测控制 MPC。在 MPC 中，在每一个时间步都会构建并求解一个凸轨迹优化问题，在一扇面向未来的滑动窗口（时域）内进行优化，并仅执行求解出的第一个控制指令。

本问题以 Dragon1 与 ISS 的对接问题为例, Dragon1 的动力学可以通过 Clohessy-Wiltshire 方程进行建模, 这是一个连续时间下的线性动力学模型。该系统的状态量和控制量如下:

$$x = [r_x, r_y, r_z, v_x, v_y, v_z]^T$$

$$u = [t_x, t_y, t_z]^T$$

其中 r 是 Dragon1 相对于 ISS 的相对位置, v 是相对速度, 而 t 是航天器受到的推力。飞船的连续时间动力学方程如下:

$$\dot{x} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} u$$

式中 $n = \sqrt{\mu/a^3}$, μ 为地球的标准引力参数, a 为国际空间站轨道的半长轴。

接下来用 LQR、凸轨迹优化和凸轨迹 MPC 三种算法来求解此问题。

3.3.1 有限时间步 LQR

取一条给定的理想参考轨迹 X_{ref} , 并使用有限时间步 LQR (FHLQR) 对其进行跟踪。

在该部分中, 控制策略为 $u_i = -K_i(x_i - x_{ref,i})$ 。

```

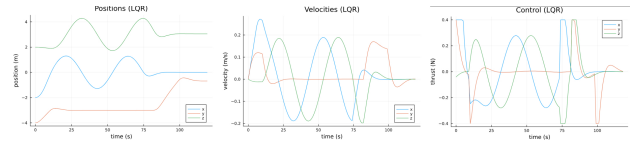
1  # create our discrete time model
2  dt = 1.0
3  A,B = create_dynamics(dt)
4  # get our sizes for state and control
5  nx,nu = size(B)
6  # initial and goal states
7  x0 = [-2;-4;2;0;0;.0]
8  xg = [0,-.68,3.05,0,0,0]
9  # bounds on U
10 u_max = 0.4*ones(3)
11 u_min = -u_max
12 # problem size and reference trajectory
13 N = 120

```

```

t_vec = 0:dt:((N-1)*dt)
X_ref = desired_trajectory_long(x0, xg, 200, dt)[1:N]
# TODO: FHLQR
Q = diagm(ones(nx))
R = diagm(ones(nu))
Qf = 10*Q
# TODO get K's from fhlqr
P,K = fhlqr(A,B,Q,R,Qf,N)
# simulation
X_sim = [zeros(nx) for i = 1:N]
U_sim = [zeros(nu) for i = 1:N-1]
X_sim[1] = x0
for i = 1:(N-1)
# TODO: put LQR control law here
    u_lqr = -K[i]*(X_sim[i] - X_ref[i])
    U_sim[i] = clamp.(u_lqr, u_min, u_max)
    # simulate 1 step
    X_sim[i+1] = A*X_sim[i] + B*U_sim[i]
end

```



通过分析仿真结果发现, 由于 LQR 代价函数只考虑惩罚偏离, 而无法理解边界, 导致飞船在对接前直接从 ISS 的实体内部横穿/撞击了过去。说明 LQR 虽然速度极快且具备闭环抗噪能力, 但面对有硬性安全边界和执行器限幅的复杂任务时会由于盲目而导致不安全的结果。

3.3.2 凸轨迹优化

假设拥有一个极其完美的动力学模型 (即不存在仿真与现实的差距), 并且拥有完美无瑕的状态估计。在此基础上, 我们将控制问题转换为一个带有诸多硬性约束的凸

轨迹优化问题进行统一求解：

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \sum_{i=1}^{N-1} \left[\frac{1}{2} (x_i - x_{ref,i})^T Q (x_i - x_{ref,i}) + \frac{1}{2} u_i^T R u_i \right] \\ \text{st} \quad & x_1 = x_{IC} \\ & x_{i+1} = A x_i + B u_i \\ & u_{min} \leq u_i \leq u_{max} \\ & x_i[2] \leq x_{goal}[2] \\ & \text{for } i = 1, 2, \dots, N \\ & x_N = x_{goal} \end{aligned}$$

在此模型中加入了初始条件约束、线性动力学等式约束、控制量的边界约束、防撞击空间站的硬性路径约束 ($x_i[2] \leq x_{goal}[2]$) 以及精确停靠的终端约束。

```

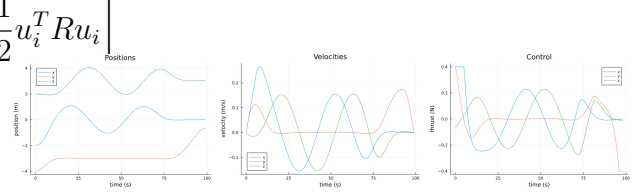
1  # cost function
2  obj = 0
3  for i = 1:N
4      obj += 0.5*cvx.quadform(X[:,i]
5      - X_ref[i],Q)
6  end
7  for i = 1:N-1
8      obj += 0.5*cvx.quadform(U[:,i]
9      ],R)
10     #obj += norm(U[:,i],1)
11 end
12 # create problem with objective
13 prob = cvx.minimize(obj)
14 # add initial condition constraint
15 prob.constraints = vcat(prob.
16 constraints, X[:,1] == x0)
17 # add terminal constraint
18 prob.constraints += X[:,N] == xg
19 # dynamics constraints
20 for i = 1:N-1
21     prob.constraints = vcat(prob.
22 constraints, X[:,i+1] == A*X[:,i] +
23 B*U[:,i])
24 end
25 # bound constraints
26 for i = 1:N
27     prob.constraints = vcat(prob.
28 constraints, X[2,i] <= xg[2])
29 end
30 for i = 1:N-1
31     prob.constraints = vcat(prob.
32 constraints, U[:,i] <= u_max)
33 end
34 prob.constraints = vcat(prob.
35 constraints, U[:,i] >= u_min)
36 end

```

```

constraints, U[:,i] >= u_min)
end

```



凸轨迹优化能够极其优雅地解决所有复杂的硬约束和安全避障问题，生成理论上的最优解。但致命缺点是它是开环的。这就导致如果系统存在仿真与现实的差距或带有噪声，会直接导致失控。

3.3.3 凸模型预测控制

在现实中，诸如不完美的状态估计、未建模的扰动动力学、推力器安装偏差以及执行器不确定性等因素，共同构成了不可忽视的“仿真与现实的差距 (Sim-to-Real Gap)”。因此，开环控制在真实世界中必然失败。

为了解决这个问题，我们引入了模型预测控制 (MPC)，它完美地将反馈控制的思想与凸轨迹优化融合在了一起。

凸 MPC 控制器在每一个时间步都会将当前的实际状态估计值作为初始条件 (x_{IC})，并重新构建和求解一个凸优化问题。不同于对长达几百秒的整条轨迹进行全局优化，MPC 只考虑一扇大小为 N_{mpc} 的向前滑动窗口。

优化问题公式如下：

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \sum_{i=1}^{N-1} \left[\frac{1}{2} (x_i - \tilde{x}_{ref,i})^T Q (x_i - \tilde{x}_{ref,i}) \right. \\ & \left. + \frac{1}{2} u_i^T R u_i \right] + \frac{1}{2} (x_N - \tilde{x}_{ref,N})^T \\ & Q (x_N - \tilde{x}_{ref,N}) \\ \text{st} \quad & x_1 = x_{IC} \\ & x_{i+1} = A x_i + B u_i \quad \text{for } i = 1, 2, \dots, N-1 \\ & u_{min} \leq u_i \leq u_{max} \quad \text{for } i = 1, 2, \dots, N-1 \\ & x_i[2] \leq x_{goal}[2] \quad \text{for } i = 1, 2, \dots, N \end{aligned}$$

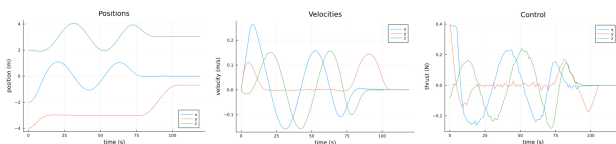
这使得 MPC 控制器能够像 LQR 一样具备前瞻性，同时通过在每一步更新状态初

值 (x_{IC}) 并让窗口向前滑动, MPC 获得了极强的动态“反应”与纠错能力, 能完美补偿和消除仿真与现实之间的差距。

```

1  # cost function
2  obj = 0
3  for i = 1:N_mpc
4      obj += 0.5*cvx.quadform(X[:,i]
5      - X_ref_window[i],Q)
6  end
7  for i = 1:N_mpc-1
8      obj += 0.5*cvx.quadform(U[:,i]
9      ],R)
10 end
11 # create problem with objective
12 prob = cvx.minimize(obj)
13 # add initial condition constraint
14 prob.constraints = vcat(prob.
15 constraints, X[:,1] == xic)
16 # dynamics constraints
17 for i = 1:N_mpc-1
18     prob.constraints = vcat(prob.
19 constraints, X[:,i+1] == A*X[:,i] +
20 B*U[:,i])
21 end
22 # bound constraints
23 for i = 1:N_mpc
24     prob.constraints = vcat(prob.
25 constraints, X[2,i] <= xg[2])
26 end
27 for i = 1:N_mpc-1
28     prob.constraints = vcat(prob.
29 constraints, U[:,i] <= u_max)
30     prob.constraints = vcat(prob.
31 constraints, U[:,i] >= u_min)
32 end

```



MPC 综合了前两者的所有长处。既拥有轨迹优化处理硬性安全约束的能力, 又通过滚动优化赋予了系统极强的闭环动态鲁棒性, 使其在未建模扰动、推力器偏差以及状态估计不完美的真实恶劣环境下能够计算出非常精确的解, 唯一缺点是计算资源消耗大, 每步都要解一次优化。